

Paper 1: GCD Analysis

10/07/26

By Royce Nogales

Table of Contents

1. **Introduction**
2. **GCD Algorithms**
 - 2.1. Euclid's Algorithm
 - 2.2. Consecutive Integer Checking
 - 2.3. Prime Factorization
3. **Analysis**
4. **Why Use Python?**
5. **References**

1. Introduction

- 1.1. This paper discusses three algorithms used to find the Greatest Common Divisor (GCD) of two numbers m and n . These algorithms are Euclid's Algorithm, Consecutive Integer Checking, and Prime Factorization. Each of these algorithms is unique in approach and use case. As well, an analysis of experimental results using the algorithms in python is also discussed.

2. GCD Algorithms

2.1. Euclid's Algorithm

- 2.1.1. This algorithm works by taking two integers m and n , and recursively passing the values of n and $m \% n$ into the function until n equals 0. This works because the divisor of (m, n) is also a common divisor of every $(n, m \% n)$.

Python

```
def euclid(m,n):  
    global intDiv  
    if n == 0:  
        print("GCD is:", m)  
        return m  
    intDiv += 1  
    ##recursively calls euclid to use find GCD of m and n.  
    return euclid(n, m%n)
```

- 2.1.2. The best case for this algorithm is $O(1)$, this is because n could be the common divisor; therefore, on the next recursive call, the GCD will be found, taking constant time. The worst case for Euclid's algorithm is $O(\log_2 m)$ because each time the $m \% n$ is taken, the result is at most be

$m/2-1$, resulting in a minimum division of 2 each time, or $\log_2 m$. This also assumes that m is the larger of the two numbers.

2.2. Consecutive Integer Checking

- 2.2.1. This algorithm works by taking the minimum number between m and n , and testing that number over and over, each time reducing it by 1, until it is a factor of both m and n .

Python

```
def consecInt(m,n):
    global intDiv
    t = min(m,n)
    ##at most, this while loop runs min(m,n)-1 times
    while t > 0:
        ##these two if statements are used to check if t is a factor of both m and
n.
        if m % t == 0: ## not using 'and' as to track integer divisions
            if n % t == 0:
                print("GCD is:", t)
                intDiv += 2
                return t
            else:
                intDiv += 2
                t -= 1
        else:
            intDiv += 1
            t -= 1
```

- 2.2.2. The best case is $O(1)$, as $\min(m,n)$ could be the GCD of the two numbers. The worst case is $O(\min(m,n))$, as the GCD of m and n could be 1, taking $\min(m,n)$ amount of time.

2.3. Prime Factorization

- 2.3.1. This algorithm works by first finding the prime factors of each number, m and n , and multiplying the common factors to find the greatest common factor. To find the prime factors, the first thing that is done is to repeatedly divide the number n by 2, while possible. Each time n is divided by 2, add 2 to an array. Then, starting from 3, ending at the square root on $n+1$, and stepping by 2 each time, n is divided by each number i repeatedly while still possible. Each time this is done, add each number i into the array. At the end, if n is still greater than 2, add n to the array. This puts all of the prime factors of n into an array. This part of the algorithm would take $O(\lg n)$ time, as the worst case is finding $\lg n$ factors of n , or all 2's.

Python

```
def primeFactorization(n):
    global intDiv
    arr = []
    ##adds all multiples of 2 from n into an array
    while n % 2 == 0:
        arr.append(2)
        n //= 2
        intDiv += 2

    ##finds all prime factors from 3 to the the square root of n
    for i in range(3, int(math.sqrt(n))+1, 2):
        while n % i == 0:
            arr.append(i)
            n //= i
            intDiv += 2

    ##if n is greater than 2, it adds that prime factor to the array
    if n > 2:
        arr.append(n)
    return arr
```

- 2.3.2. The second part of the algorithm requires comparing the arrays of m and n for common integers. This is done by using nested loops of length $\lg m$ and $\lg n$, as that is how many integers are in the arrays of m and n . It compares the i th element of m to the j th element of n ; if there is a match, the element is removed from n and i is incremented. The removed element is multiplied to an integer ret , which has an initial value of 1. At the end of this function, the value ret is returned. The time of this algorithm is $O(\lg m * \lg n)$, as it has a loop m and a nested loop n of lengths $\lg m$ and $\lg n$ respectively.

Python

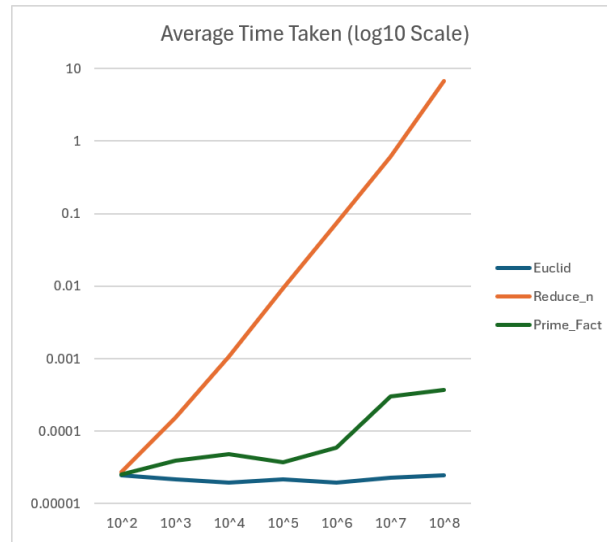
```
def commonFactors(m,n):
    ret = 1
    for i in range(len(m)):
        j = 0
        while j < len(n):
            if(m[i] == n[j]):
                ret *= n.pop(j)
                j = len(n)
            j += 1

    return ret
```

2.3.3. Overall, this algorithm would take a time of $\lg n + \lg m + (\lg m * \lg n)$, which ultimately reduces down to $O(\lg m * \lg n)$.

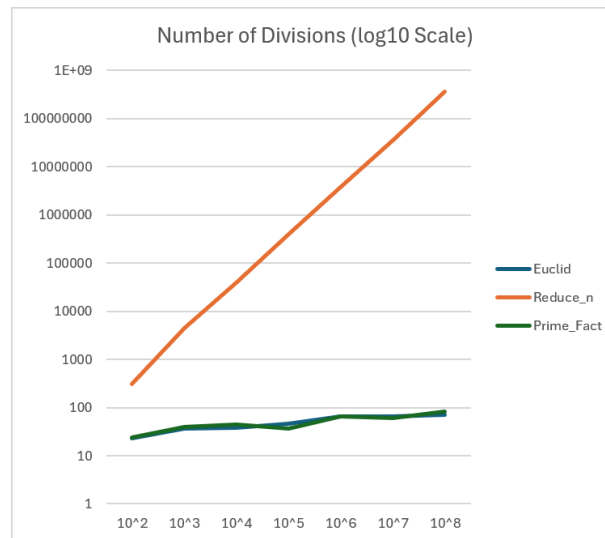
3. Analysis

3.1. Graph 1: Average Time Taken



This shows that the average time taken for the Reduce_n algorithm (Consecutive Integer Checking) increases at a linear rate, as both of the axes are $\log_{10}$. It also shows that Euclid stays constant, while Prime Factorization increases very slowly. Compared to the Big Omega times discussed previously, the Reduce_n algorithm makes sense, as it is linear, Prime factorization is close to being $\lg m * \lg n$, while Euclid's algorithm is very constant with the time, showing almost $O(1)$ time. This shows that the Reduce_n performed as expected, the Prime Factorization performed slightly better than expected, and Euclid's algorithm performed much better than expected.

3.2. Graph 2: Total Number of Integer Divisions



This graph shows the number of integer divisions done by each algorithm for each set of times it is run. This graph pretty closely matches with the previous graph showcasing the Average time it took to complete each algorithm.

Reduce_n took a linear amount of divisions, while Euclid's algorithm and the Prime Factorization took close to $3 \cdot \log_2(\text{the scale of the numbers } m \text{ and } n)$, which is as to be expected.

3.3.

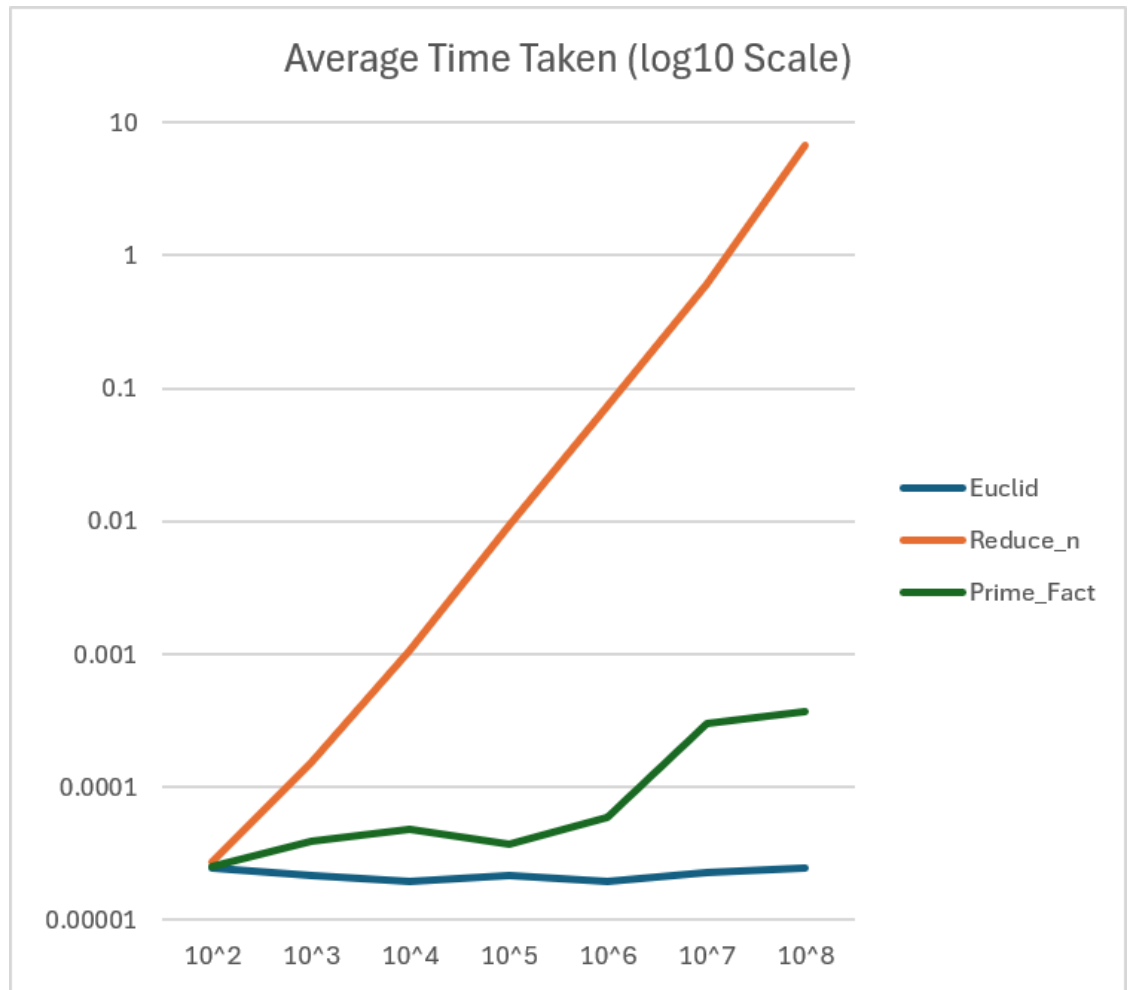
4. Why Use Python?

- 4.1. The choice of Python for the programming language used was because of the 'timeit' library. This allowed me to run a function n amount of times, and it would give the total amount of time from start to finish of all n times it ran. I am not sure of other features that would have sped up the execution time of my functions.

5. References

- 5.1. <https://www.geeksforgeeks.org/python/python-program-for-efficient-program-to-print-all-prime-factors-of-a-given-number/>

5.2.



5.3.

